Architecture 1001:
x86-64 Assembly
Mystery Lystery 2 Solved!

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- **What's up with the sub/add 0x28 in main()?**
- Why is VS over-allocating space for a single local variable?



Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- **What's up with the sub/add 0x28 in main()?**
- Why is VS over-allocating space for a single local variable?



CallASubroutine1.c

```c
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
0000000140001000  mov          eax,0BEEFh
0000000140001005  ret
main:
0000000140001010  sub          rsp,28h
0000000140001014  call         func (0140001000h)
0000000140001019  mov          eax,0F00Dh
000000014000101E  add          rsp,28h
0000000140001022  ret
```

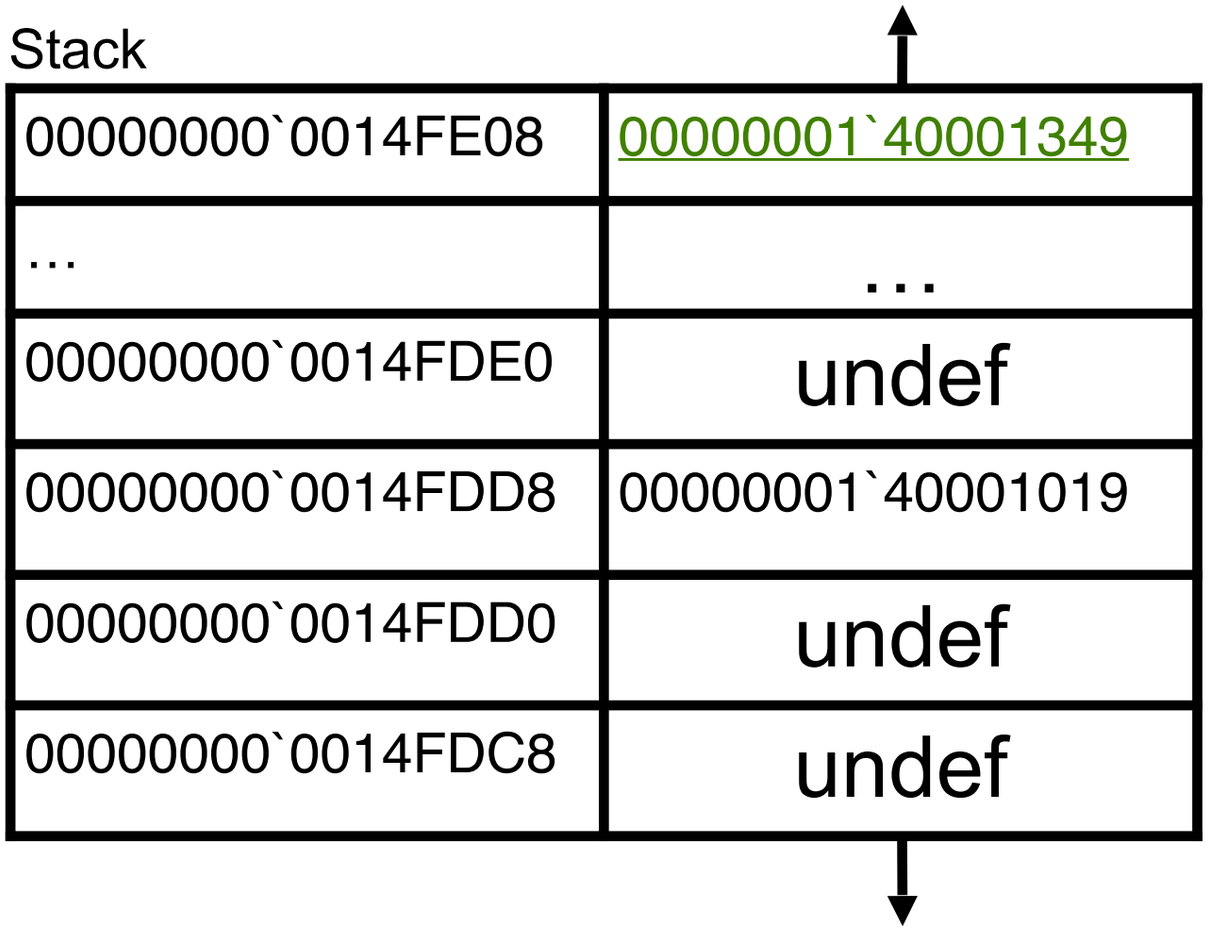
CallASubroutine1.c: STACK FRAME TIME OUT

```
func:
mov    eax,0BEEFh
ret
main:
sub    rsp,28h
call   func (0140001000h)
mov    eax,0F00Dh
add    rsp,28h
ret
```

"Function-before-main""s frame

main's frame

func never even makes a stack frame



CallASubroutine1.c:
STACK FRAME TIME OUT

```
func:
mov    eax,0BEEFh
ret
main:
sub    rsp,28h
call   func (0140001000h)
mov    eax,0F00Dh
add    rsp,28h
ret
```

"Function-before-main""s frame

main's frame

0x28 bytes

func never even makes a stack frame

Stack

00000000`0014FE08	00000001`40001349
00000000`0014FE00	*16-byte-stack-alignment padding*
00000000`0014FDF8	r9 shadow store
00000000`0014FDF0	r8 shadow store
00000000`0014FDE8	rdx shadow store
00000000`0014FDE0	rcx shadow store
00000000`0014FDD8	00000001`40001019
00000000`0014FDD0	undef
00000000`0014FDC8	undef


SEEMINGLY USELESS OVER-ALLOCATION OF
SPACE IN FUNCTIONS THAT CALL FUNCTIONS

MICROSOFT "x64" ABI
SHADOW SPACE



Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- **What's up with the sub/add 0x28 in main()?**
- Why is VS over-allocating space for a single local variable?



Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- **What's up with the sub/add 0x28 in main()?**
- Why is VS over-allocating space for a single local variable?

SOLVED!

SOLVED!